Onboarding Vault: protecting new-employee data with AES, DES and RC4

Information Security, Assignment 1 · Group: [Name 1, NRP] · [Name 2, NRP] · [Name 3, NRP] · [Name 4, NRP] · Code: github.com/mikal-zh/info_security

1. Scenario and why the application is useful

A 200-employee logistics company in Surabaya onboards about 15 people a month. Each new hire must hand HR a **KTP scan**, a **signed contract and CV** (Word/PDF), a **recorded interview video**, and a form with NIK, NPWP, salary bank account, BPJS number, health details and dependants. Today this travels through e-mail and a shared drive, where any IT administrator, backup operator or thief of a laptop or disk can read it. **Users** are the employees, each of whom logs in with their employee number and stores and retrieves only their own file. The application keeps all of it encrypted at rest with keys that only the employee's password can unlock. The threat it addresses is **theft or exposure of the stored data** (database dump, disk, backup, curious administrator). This matters legally: GDPR Art. 32(1)(a) and UU PDP Pasal 35 require security measures such as encryption, and GDPR Art. 34(3)(a) exempts the controller from notifying every data subject after a breach if the data was unintelligible. A leak of plaintext would instead trigger the 3×24-hour notification of UU PDP Pasal 46.

2. What counts as private

Every item identifies the employee, so **everything the employee submits is personal data and is encrypted**, including file names, types and sizes. We still classify items, because the specific ones carry higher risk and would require a DPIA (UU PDP Pasal 34):

Specific (UU PDP Pasal 4(2) / GDPR Art. 9)	General (UU PDP Pasal 4(3) / GDPR Art. 4(1))	Stored in clear, and why
Blood type, allergies (health, 4(2)a); salary account, payslips, contract salary (financial, 4(2)f); dependants' data (children, 4(2)e); **religion**: general under UU PDP 4(3)d but special under GDPR Art. 9(1), so we apply the stricter rule.	Name, NIK (also GDPR Art. 87), birth place/date, gender, nationality, marital status, address, phone, e-mail, NPWP, BPJS number, emergency contact (a third party's data), KTP scan, interview video. A face photo or video is *not* biometric data here, because it is never processed for unique identification (GDPR Art. 4(14), Recital 51).	Employee number (needed to log in, not secret), document category, timestamps, ciphertext lengths. None reveals more than "an employee uploaded a document of this kind".

3. Design decisions

- **Architecture.** Flask server with SQLite for rows and one file per ciphertext on disk. Encryption is done on the server rather than in the browser, so that the same code path stores and verifies all three ciphers and HR could later be given audited access. The cost is that the server holds keys while a user is logged in, so keys are kept only in memory (below).
- **All three ciphers, every time.** Each field, file and file-metadata record is sealed separately with AES, DES and RC4, and all three copies are stored. The employee can download any copy, and "Verify all 3" checks every copy. This meets "everything stored is protected with each cipher" literally, and it means the comparison in §4 runs on the same real data. The consequence, stated plainly: **confidentiality is only as strong as the weakest copy (DES)**; see §5.
- **Modes.** AES-256 and DES both use **CBC with a fresh random IV per message** (16 or 8 bytes, stored in front of the ciphertext) and PKCS#7 padding. ECB is excluded because it leaks equal blocks, which is visible in KTP images and in repeated form values. We chose CBC over CTR because a repeated CTR nonce reveals the XOR of two plaintexts, and DES's 64-bit block leaves too little room for random nonces plus counters. A repeated CBC IV only reveals whether two messages start with the same blocks. GCM is defined only for 128-bit blocks, so it cannot be used with DES. We use one construction for all ciphers so that they are compared fairly.
- **RC4.** RC4 has no IV, and reusing one key for all records (the bug in our first version) reuses the keystream, so XORing two ciphertexts gives the XOR of the two plaintexts. Each message now uses a key HMAC-SHA256(k_{RC4}, 16-byte random nonce), and the first 3072 keystream bytes are discarded, because RC4's early output is biased.
- **Integrity: "comes back intact".** CBC and RC4 are malleable: flipped ciphertext bits produce controlled plaintext changes. Every envelope `version|cipher|IV|ciphertext` therefore carries an **HMAC-SHA256 tag (encrypt-then-MAC)** with a separate MAC key per cipher. The tag also covers a context string such as `user:7|doc:<uuid>|blob`, so a ciphertext copied into another user's row fails the check. The tag is checked *before* unpadding, which rules out padding-oracle attacks. Files are also compared with a SHA-256 kept inside the sealed metadata. A copy that fails is never sent to the user, which our tests check by flipping one byte.
- **Key handling.** Password + 16-byte per-user salt → PBKDF2-HMAC-SHA256 with 600,000 iterations (OWASP 2023). From this we derive a login verifier (only its hash is stored, and it is compared in constant time) and a key-encryption key. A random 256-bit **vault key** per user is stored only wrapped with AES-256-GCM under that key. Per-cipher encryption keys (256-bit AES, 56-bit-effective DES, 128-bit RC4) and MAC keys are derived from the vault key by HMAC with labels. As a result, a password change re-wraps 32 bytes instead of re-encrypting gigabytes, and identical passwords never give identical keys. There is no recovery without the password: that is the price of administrators not being able to read the data.

- **Sessions.** Our first version kept the password in Flask's cookie, which is signed but *not encrypted*. Now the cookie holds only a random token, and the unwrapped keys stay in server memory and are removed on logout or after 30 minutes idle. Every POST is CSRF-protected, and ownership is enforced in every SQL query (another user's document returns 404).
- **Storage.** Ciphertexts are named `<random uuid>.aes|.des|.rc4`, so neither names nor owners appear on disk. Account erasure (GDPR Art. 17 / UU PDP Pasal 8) deletes all rows and files.

4. Evaluation: what we measured and why

What matters to employees is the time to **upload and retrieve their own items**. What matters to the company is **storage cost**. So we timed the operations the app really performs (cipher alone, and the full seal/open with HMAC) on **workloads sized like the real items**: one 16 B field, the 17-field form, a 400 KB KTP scan, a 2 MB contract and a 30 MB (≈2 min, 720p) interview video. We also timed a whole HTTP download through the app and the PBKDF2 login, to put the cipher in proportion. Inputs are random bytes, because cipher speed does not depend on content. Each figure is the median of 51 runs (15 for the video) after warm-up, with GC off and the process pinned to one performance core. Interquartile ranges were below 1–2 % of the median. Between two full runs the video download varied by about 20 % (207 vs 248 ms for AES), but the order of the ciphers never changed. Platform: i7-1260P with AES-NI, on battery power, Python 3.13, PyCryptodome 3.24 (`benchmark/run_benchmark.py`; raw data in `benchmark/results/`).

Median time (ms)	**AES-256-CBC**	**DES-CBC**	**RC4-drop3072**
One field, seal (incl. HMAC)	0.013	0.017	0.028
Full form, 17 fields, seal	0.22	0.29	0.47
KTP 400 KB, decrypt only / open	0.8 / 1.2	6.5 / 7.0	1.8 / 2.5
Video 30 MB, decrypt only / open	61 / 99	502 / 539	135 / 204
Video 30 MB, seal (upload)	105	644	173
Raw cipher throughput, decrypt	≈ 500 MB/s	≈ 60 MB/s	≈ 223 MB/s
Video download, end-to-end in app	207–248	651–693	283–324
Size overhead per item (header+IV+padding+tag)	51–66 B	43–50 B	50 B

Interpretation. (1) **AES is the fastest, not RC4**: on a CPU with AES-NI, AES runs in hardware, while RC4 processes one byte at a time in software. DES, which uses bit permutations and has no hardware support, is 8× slower than AES. (2) For small items the order is set by **fixed per-message cost**, not throughput. RC4 is slowest on a single field because each message pays for deriving a fresh key, the key schedule and 3072 discarded bytes. Saving the whole form with all three ciphers still takes about 1 ms, which is invisible next to the 196 ms PBKDF2 login. (3) Integrity costs about as much as AES itself on large files: HMAC-SHA256 adds about 38 ms per 30 MB. We keep it, because returning a tampered file would break the "intact" requirement. (4) **None of these numbers limits the user**. Sending a 30 MB video over a 20 Mbit/s office link takes about 12 s, so even DES decryption (~0.5 s) is under 5 % of a download and AES under 2 %. Uploading the video under all three ciphers takes about 0.9 s, 70 % of it DES. (5) Storage overhead per item is at most 66 B, which is negligible. The real cost is keeping three copies: 3× storage, about 100 MB per employee or 20 GB for 200 employees. **Conclusion for the scenario:** performance does not decide between the ciphers; security does. AES-256 is both the fastest and the only one still considered secure.

5. Limitations and what we would change in production

DES's 56-bit key was broken by exhaustive search in 56 hours already in 1998 (EFF Deep Crack), and file headers such as `%PDF` or the PNG signature give the known plaintext an attacker needs. So anyone with a copy of the disk could recover one employee's DES key and read that employee's DES copies. The other subkeys stay safe, because they are derived independently from the vault key. RC4's keystream biases led RFC 7465 to ban it from TLS. Keeping DES and RC4 copies is therefore required by the assignment, not a security choice: a production system would store only the AES copy (or use AES-GCM). Further gaps: HTTPS must be added in deployment; a compromised running server can read the keys of logged-in users; and HR access would need an explicit, audited sharing step, which is out of scope here.